

从 SGD 开始理解现代神经网络优化器

第一讲：随机梯度下降的三种身份

这份讲义的目标不是背公式，而是建立一个会反复使用的框架：优化器每一步到底在选择什么方向，如何缩放这个方向，以及随机噪声如何改变训练动力学。

本讲要回答

- 为什么 SGD 是全量梯度下降的随机近似？
- mini-batch 梯度为什么可以看作带噪声的真梯度？
- 学习率为什么被曲率限制？
- 为什么这会自然引向 Momentum、Adam 和 Muon？

本讲暂不做

- 不直接证明 Muon 的完整性。
- 不把所有优化器混在一起讲。
- 先把 SGD 作为基准坐标系立住。

建议读法：把每一页当成课堂板书，先看公式，再看公式下面的问题。

训练到底在最小化什么？

设模型参数为一个向量。现在先不关心它来自线性层、卷积层还是 Transformer，只把所有参数摊平成：

$$\theta \in R^d$$

理论上，我们希望最小化总体风险，也就是对真实数据分布的期望损失：

$$L(\theta) = E_{z \sim D}[\ell(\theta; z)]$$

符号说明

- $L(\theta)$ ：模型在真实数据上的平均损失。
- θ ：模型参数，可以先理解成所有权重摊成的向量。
- z ：一个训练样本，通常包含输入和标签。
- D ：真实数据分布，也就是样本从哪里来。
- $\ell(\theta; z)$ ：参数为 θ 时，在单个样本 z 上的损失。
- $E_{z \sim D}[\dots]$ ：对从 D 中抽到的样本取平均。

现实中我们只有训练集，所以常用经验风险：

$$L_N(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(\theta; z_i)$$

课堂提醒

优化器真正看到的不是数据分布，而是训练过程中不断抽到的 mini-batch。SGD 的随机性就是从这里来的。

全量梯度下降太贵，SGD 是它的廉价估计

如果每一步都看完整训练集，得到的是全量梯度下降：

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t)$$

SGD 改成只看一个 mini-batch。令这一批样本的索引集合为：

$$B_t \subset \{1, \dots, N\}, \quad |B_t| = B$$

mini-batch 梯度定义为：

$$g_t = \frac{1}{B} \sum_{i \in B_t} \nabla_{\theta} \ell(\theta_t; z_i)$$

于是 SGD 的一行更新就是：

$$\theta_{t+1} = \theta_t - \eta_t g_t$$

```
for t = 0, 1, 2, ...  
  sample a mini-batch B_t  
  compute gradient estimate g_t  
  update theta <- theta - eta_t * g_t
```

SGD 的核心性质：无偏但有噪声

在常见的均匀采样假设下，mini-batch 梯度是全量梯度的无偏估计：

$$E[g_t | \theta_t] = \nabla L(\theta_t)$$

所以我们可以把它拆成真梯度加噪声：

$$g_t = \nabla L(\theta_t) + \xi_t, \quad E[\xi_t | \theta_t] = 0$$

把这个拆分代回更新式：

$$\theta_{t+1} = \theta_t - \eta_t \nabla L(\theta_t) - \eta_t \xi_t$$

确定性部分

沿着负梯度方向下降，体现优化目标的几何。

随机性部分

每一步都注入噪声，影响探索、稳定性和泛化行为。

这就是 SGD 的第二个身份：它不是单纯的下降算法，而是一个带噪声的离散动力系统。

学习率不是旋钮，它受曲率约束

假设损失函数是 β -smooth，也就是梯度变化不太剧烈。一个常用的不等式是：

$$L(\theta - \eta g) \leq L(\theta) - \eta \nabla L(\theta)^T g + \frac{\beta \eta^2}{2} \|g\|^2$$

当 g 等于完整梯度时，下降项和二次惩罚项之间有一个竞争关系：

$$L(\theta_{t+1}) \leq L(\theta_t) - \eta \left(1 - \frac{\beta \eta}{2}\right) \|\nabla L(\theta_t)\|^2$$

关键解释

学习率太小，下降慢。学习率太大，二次项会压过下降项，更新可能震荡甚至发散。
曲率越大，可接受的学习率越小。

SGD 更复杂，因为 g 还有噪声。粗略地说，学习率同时控制两件事：

- 向负梯度方向走多远。
- 把梯度噪声放大多少。

为什么病态曲率会拖慢 SGD?

看最经典的二次函数：

$$L(\theta) = \frac{1}{2} \theta^T H \theta$$

它的梯度是：

$$\nabla L(\theta) = H\theta$$

全量梯度下降变成一个线性系统：

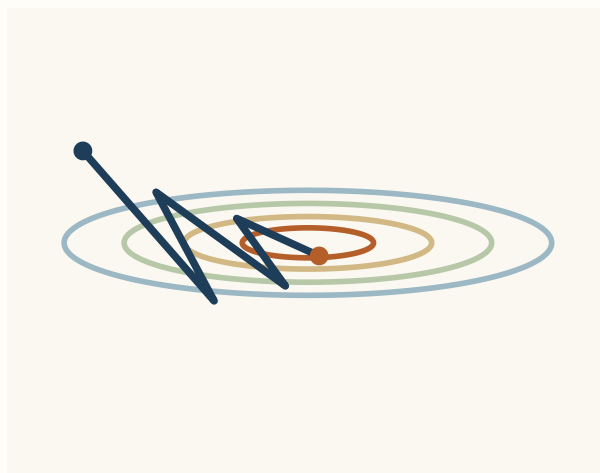
$$\theta_{t+1} = (I - \eta H) \theta_t$$

若 H 的特征值为 λ_i ，那么每个特征方向上的误差满足：

$$e_{t+1}^{(i)} = (1 - \eta \lambda_i) e_t^{(i)}$$

稳定性要求大致为：

$$0 < \eta < \frac{2}{\lambda_{\max}}$$



狭长等高线会让梯度方向来回摆动。

条件数 $\kappa = \lambda_{\max} / \lambda_{\min}$ 越大，陡峭方向限制学习率，平坦方向却收敛很慢。

batch size 改变的是噪声，不只是速度

若单样本梯度的协方差约为 Σ ，那么 mini-batch 平均以后，梯度估计的协方差近似缩小为：

$$\text{Cov}\left(g_t \mid \theta_t\right) \approx \frac{\Sigma}{B}$$

所以 batch size B 增大时，梯度估计更稳定，但每一步的随机扰动也更小。

小 batch

- 单步便宜。
- 噪声较大。
- 可能帮助跳出狭窄区域。

大 batch

- 梯度更准。
- 并行效率更高。
- 学习率和 warmup 更敏感。

把噪声拆分代入更新式，可以看到噪声被学习率 η 放大：

$$\text{noise in update} \approx -\eta \xi_t, \quad \text{Var}\left(\eta \xi_t\right) \propto \frac{\eta^2}{B}$$

这就是为什么学习率和 batch size 不能完全分开调。它们共同决定训练动力系统的温度。

SGD 只做了两件事：方向和标量步长

SGD 的更新可以读成一句话：

$$\text{new parameter} = \text{old parameter} - (\text{scalar}) \times (\text{gradient})$$

它没有显式回答这些问题：

- 1. 不同坐标是否应该用不同步长？
- 2. 过去许多步的梯度方向是否应该累积？
- 3. 矩阵参数是否应该保留矩阵结构，而不是摊平成向量？
- 4. 更新方向是否应该被归一化、白化或正交化？

问题	后续优化器的回答
单步梯度太吵	Momentum 使用时间平均。
不同坐标尺度不同	RMSProp、Adam 使用坐标级自适应缩放。
矩阵方向有结构	Shampoo、SOAP、Muon 等方法显式处理矩阵几何。

所以 SGD 是所有现代优化器的零点：后面的每个方法，都可以看成对 g_t 或 η_t 的改造。

下一步：把单步梯度变成时间方向

SGD 每一步只相信当前 mini-batch 的梯度。Momentum 的出发点很朴素：如果很多步的梯度都大致指向同一方向，那这个方向更可信。

Momentum 引入速度或动量变量：

$$v_t = \mu v_{t-1} + g_t$$

$$\theta_{t+1} = \theta_t - \eta v_t$$

在低噪声方向

梯度反复同向，动量会累积，走得更快。

在震荡方向

梯度来回变号，动量会互相抵消，震荡被削弱。

这一步非常重要，因为 Muon 也会先形成一个带动量的矩阵信号，再对这个矩阵信号做几何处理。

single gradient → momentum signal → structured update

从向量参数到矩阵参数

神经网络里很多核心参数天然就是矩阵，比如线性层权重：

$$W \in \mathbb{R}^{m \times n}$$

如果只照搬 SGD，就是：

$$W_{t+1} = W_t - \eta G_t, \quad G_t = \nabla_W L(W_t)$$

Muon 这条线的核心转弯是：先把梯度或动量看作矩阵，再对这个矩阵方向做正交化或极分解近似，得到结构化更新。

1. 梯度矩阵

当前 batch 给出 G_t 。

2. 动量矩阵

累积成 M_t ，降低单步噪声。

3. 几何整理

把 M_t 转成更规整的矩阵更新 U_t 。

$$M_t = \mu M_{t-1} + G_t, \quad U_t \approx \text{Orth}(M_t)$$

$$W_{t+1} = W_t - \eta U_t$$

这只是预告，不是完整定义。下一讲先把 Momentum 讲透，再继续到自适应方法和矩阵预条件。